

DATUM Backend Performance — Status Report

Analytics service, tenant databases, and ETL pipeline · Prepared 2026-08-30 · Updated 2026-08-31

Why the dashboard slows down for larger brands, what's already fixed, what's left, and what needs to move upstream into the ETL pipeline.

Method: live timing and query-plan checks against the tenant databases (8 brands sampled, ranging from small to the largest current brand), plus a direct read of the exact backend code behind each dashboard feature.

Found: 11 issues/changes tracked — 7 fixed and verified, 1 query-level fix verified but not yet resolving the end-to-end symptom (item 8), 1 fixed for request count but still slow per-query (Payment Split), 1 still open (Product Conversion's unbounded scan), plus one attempted index that was verified useless and cleanly removed.

Important correction: item 8 below fixed one specific, real, verified database problem (a missing index behind the KPI grid's own query) — but **the dashboard is still not fixed**. Real-world testing shows the full page still takes **30-35 seconds to load** for big brands at 30+ day ranges. The isolated query fix is confirmed correct; it just isn't the whole story. See item 8 for what's confirmed vs. what's still unexplained.

1. Issues Found

1. Sales trend charts were scanning the entire orders table FIXED

Every time the dashboard drew a daily or hourly sales/orders trend chart, the database had to check every single order row in the date range — up to ~2 million rows for the largest brand — because the column being counted (`order_name`) wasn't part of any database index. For the largest brand this took close to **2 minutes** per load.

```
analytics/services/metricsSnapshotService.js:1474 fetchHourlyRows
```

```
analytics/services/metricsSnapshotService.js:1650 fetchDailyRows
```

```
analytics/services/metricsAggregateService.js:298 queryOrderSalesTotals
```

2. Product list page could fetch an unlimited number of rows FIXED

When sorting the product list by stock-based metrics ("days of stock", "daily run rate") or exporting it to CSV, the query that pulled candidate rows had no upper limit at all — for a store with a very large catalog, this could pull back an unbounded number of rows in one request before the app even started sorting them.

```
analytics/services/productConversionService.js:1005 getProductConversion (inventory-sort path)
```

```
analytics/services/productConversionService.js:1037 getProductConversionCsv
```

```
analytics/services/productConversionService.js:390 buildSelectSql
```

3. Returns numbers ran two database trips instead of one FIXED

Getting "this period" and "last period" counts of cancelled/refunded/returned orders made two separate calls to the database instead of one combined request.

```
analytics/services/metricsSnapshotService.js:863 getReturnsSnapshotPair
```

```
analytics/shared/utils/metricsUtils.js:23 computeReturnCounts
```

4. Dashboard summary loads its pieces one at a time

FIXED — DIFFERENT CAUSE THAN FIRST THOUGHT

The main dashboard KPI summary is built from up to 8 independent pieces of data, fetched one after another. The original theory was that overlapping these 5-8 pieces (parallelizing) was the fix. Live measurement disproved that: it wasn't 8 evenly-slow pieces — **2 of the 6 branches that actually ran accounted for 97% of the total time** (19.2s and 11.9s out of ~32s on TMC), and both traced back to one missing index (see item 8 below). Parallelizing alone would have capped the improvement at best-case ~19s; fixing the real cause got it to 2.4s. Kept here as a record of the corrected diagnosis — the actual fix is filed under item 8.

```
analytics/services/metricsSnapshotService.js:2286 getDashboardSummary
```

```
analytics/shared/db/tenantConnection.js:9 POOL_MAX (default 5)
```

```
analytics/shared/db/tenantConnection.js:16 poolKey
```

5. Payment Split Trend chart makes far too many requests

REQUEST COUNT FIXED — QUERY ITSELF STILL SLOW

To draw one "payment split over time" chart, the dashboard used to call the backend once per hour or once per day (up to ~360 calls for a 90-day view). **Fixed:** a new bucketed endpoint returns the whole series in one request — 90-day view now 2 requests instead of ~360; the Mode of Payment donuts went from 4 requests to 1. **Still open:** the underlying query itself remains slow (measured 72-90 seconds on TMC for a 30-day view) because it classifies each order's payment type with a text-search (LIKE '%Gokwik PPCOD%', etc.) against a raw text column — a pattern no database index can speed up, since it has to read and pattern-match every row's actual text. An index was attempted and confirmed *not* to help (MySQL never chose to use it — see item 11), then removed. The real fix needs a pre-computed column (see below), not yet implemented.

```
analytics/services/metricsReportService.js:242 getPaymentSalesSplit
```

```
analytics/services/metricsReportService.js:440 getOrderSplit
```

```
analytics/services/metricsReportService.js (new) getPaymentSplitTrend, getPaymentSplitSummary
```

```
client/dashboard/src/features/dashboard/widgetDataHooks.js:1238
useDashboardPaymentSplitTrendData
```

```
client/dashboard/src/components/HourlySalesCompare.jsx:974 (same pattern, not yet fixed)
```

6. Product list page repeats the same groundwork twice FIXED

Loading a page of the product conversion table ran two separate queries back to back — one to count total matching products, one to fetch the actual page — both independently redoing the same underlying joins. **Fixed** by folding the count into the page-fetch query using a window function (`COUNT(*) OVER()`), now supported since the database version was confirmed to support it (MySQL 8.0.45). The now-unused separate count-query builder was removed. Verified via updated unit tests (6/6 passing); a live production timing comparison hasn't been captured yet.

```
analytics/services/productConversionService.js:401 buildSelectSql (now builds COUNT(*) OVER())
```

```
analytics/services/productConversionService.js:978 getProductConversion (single query)
```

7. Notifications feature fetches its full recipient list every time LOW PRIORITY

Sending a push notification fetches every stored device token with no filter or limit. Currently harmless — only 16 records exist in this collection today — so this is flagged for later, not urgent now.

```
alerts-service/utils/fcm.js:45 sendToAll
```

```
alerts-service/utils/fcm.js:60
```

8. KPI grid's "today vs. same completed hours yesterday" comparison had no index either

QUERY-LEVEL FIX VERIFIED — DASHBOARD STILL SLOW END-TO-END

Exact issue: whenever the selected date range includes today (true for almost any real selection), the KPI grid runs an extra comparison so a partial "today" isn't unfairly compared against a full previous day. That comparison calls the same order-totals query as item 1 — but with one extra condition, `AND created_time < ?`. The item-1 index didn't include that column, so the database still had to look up the full row for every matching order just to check that one condition. This one query shape gets called **3 times** across two of the summary's branches.

Fix applied: new covering index (`created_date, created_time, order_name, total_price`) on `shopify_orders`, applied live and online (`ALGORITHM=INPLACE, LOCK=NONE`, no write blocking) across all 8 brands, each verified with a real before/after query plan, timing, and byte-identical result diff — then re-confirmed against the actual read replica the live app queries (not just the write database), with zero replication lag.

Brand	Before	After	Speedup
TMC	24.0s	1.2s	19.7x
PTS	5.4s	0.4s	12.0x

BBB	20.9s	0.5s	38.4x
AJMAL	16.5s	0.5s	35.8x
VAMA	6.8s	0.4s	15.8x
URBAN HINDU	0.2s	0.1s	2.0x
3TENX	0.5s	0.2s	2.4x
SHYLENEW	0.5s	0.1s	5.1x

This specific query, isolated (TMC, full KPI grid, all branches combined): 32-44 seconds → 2.4 seconds — measured by calling the backend function directly, bypassing the network/HTTP layer.

Not fixed — the actual dashboard page is still slow. Real-world browser testing after this fix shipped shows the full page still takes **30-35 seconds** to load for big brands at 30+ day ranges. The isolated-query improvement above is real and verified, but it has not translated into a faster page. Leading theory, not yet confirmed: the dashboard fires ~15-20 requests at once on load, several of which (particularly Payment Split — still 72-90s per query, see item 5) share the same brand's database connection pool, capped at 5 connections — so this now-fast query may simply be queueing behind other still-slow ones rather than running slowly itself. This needs direct confirmation (checking actual connection-pool wait time under real concurrent load) before it can be called understood, let alone fixed.

```
analytics/services/metricsAggregateService.js:298 queryOrderSalesTotals
```

```
analytics/services/metricsSnapshotService.js:2311 getDashboardSummary (totalPair/deltaPair)
```

```
analytics/services/metricsSnapshotService.js:1241 getRowTwoComparisonSnapshots
```

9. Duplicate network request for page-speed data FIXED

The Web Vitals panel and the Web Performance panel both independently fetched the exact same page-speed data (same brand, same date range) — two identical requests every dashboard load, because the fetch bypassed the shared request cache the rest of the dashboard uses. Fixed with a small shared cache so whichever widget asks first triggers the real fetch and the second one reuses it.

```
client/dashboard/src/features/dashboard/widgetDataHooks.js fetchPagespeedResults
```

10. DRR/DOH sorting removed from Product Conversion and Inventory tables

DONE (BY REQUEST, NOT A PERF BUG)

Sorting by DRR ("daily run rate") or DOH ("days of stock") required fetching every matching row and sorting in application code (these values come from Redis, not the database, so the database can't sort by them). Removed the sort affordance from both tables' column headers since it wasn't needed in practice — filtering by DRR/DOH is unaffected and still works.

```
analytics/services/productConversionService.js ALLOWED_SORT, needsInventoryPostProcessing
```

```
client/dashboard/src/components/ProductConversionTable.jsx, InventoryTable.jsx
```

11. Investigated: a payment-gateway index — tried, measured, found useless, removed

REVERTED, DOCUMENTED

Before landing on the pre-computed-column plan for item 5, a more direct index was tried first: (created_date, payment_gateway_names(191), order_name, total_price). Applied to TMC, it only improved timing by ~25% (90s → 68s) and the query plan showed MySQL never actually chose to use it (key: null) — confirming a text-search index limitation, not a fluke. Rather than leave a useless index quietly costing write overhead forever, it was dropped from TMC (the only brand it was ever applied to) once confirmed. No other brand was ever touched by this one.

2. Fixable Directly in the Backend Service Code

These don't require any change to the ETL pipeline or the raw database tables — just changes to how the analytics service queries and combines data.

Issue	What changes	Status
#1 — Sales trend scans	Added a database index covering created_date, order_name, total_price on shopify_orders, rolled out to all 8 brand databases.	Done
#2 — Unbounded product export	Added a safety row limit (UNBOUNDED_ROW_SAFETY_CAP) to the two paths that had no ceiling, sized well above real catalog sizes measured live across all 8 brands.	Done
#3 — Returns double-query	Combined the two returns-count queries into one, using a single query with conditional sums for current vs. previous period (computeReturnCountsPair).	Done
#4 — Dashboard summary serial loading	Turned out not to be the cause on its own — see #8. Parallelization not implemented; the index fix in #8 alone didn't resolve the dashboard's real-world load time, so this may still need revisiting once #8's remaining gap is understood.	Not resolved — see #8
#5 — Payment Split Trend (backend half)	Added a bucketed endpoint (getPaymentSplitTrend) and a consolidated summary endpoint (getPaymentSplitSummary) so requests collapse from ~360/4 down to 1-2. The underlying query is still slow per-call (see item 5, item 11) — needs a pre-computed classification column next.	Request count done; query speed still open
#6 — Product list double query	Merged the count query into the row-fetch query via COUNT(*) OVER(), now that the database version is confirmed to support it (MySQL 8.0.45).	Done

#7 — Notifications full fetch	Add a filter/limit if the recipient list ever grows meaningfully. Not scheduled — low impact today.	Backlog
#8 — KPI grid missing created_time index	New covering index on shopify_orders, all 8 brands. Isolated query timing: TMC 32-44s → 2.4s. Real dashboard page load is still 30-35s for big brands at 30+ day ranges — this fix did not resolve the reported symptom on its own; likely masked by connection-pool contention from other still-slow widgets, not yet confirmed.	Query fixed; symptom not resolved
#9 — Duplicate page-speed request	Shared request cache added so Web Vitals and Web Performance panels reuse one fetch instead of two.	Done
#10 — DRR/DOH sorting	Removed from both tables' UI and backend allow-list, per product decision (filtering unaffected).	Done
#11 — Payment-gateway text index (attempt)	Applied to TMC, measured, confirmed unused by the query planner, removed.	Reverted
New — Payment Split classification column	Pre-compute COD/Prepaid/Partial as a small generated column at write time (MySQL feature, auto-maintained on every insert/update — no pipeline changes needed), index that instead of the raw text, update the 4 query functions to read it.	Designed, not yet built
New — Product Conversion unbounded scan	product_variant_map CTE scans the brand's entire order history with no date bound, every load, regardless of selected range. Needs a date bound or its own index; not yet designed.	Open, not started

3. Needs a Pipeline-Side Fix (New or Updated Rollup Tables)

A few of these issues are patched for now but will keep getting slower as raw order/session data keeps growing, because the backend still has to add up raw rows at the moment a dashboard request comes in. The lasting fix for those is moving the aggregation earlier — into the ETL pipeline — so the backend can read a small, pre-summed table instead.

Add a daily order rollup table PIPELINE

A table with one row per day (plus whichever filter breakdowns the dashboard supports — UTM source, device, city, product) holding pre-added-up order counts and sales. This would let the daily/hourly trend charts (issue #1) read a few dozen rows instead of scanning the full orders table — a fix that keeps working no matter how much order data piles up, unlike the index fix which only buys time.

Payment-split fix update: turned out to be a database change, not a pipeline change

CORRECTION

Originally scoped as a pipeline rollup table. After investigation, the actual bottleneck is a text-search classification (LIKE '%Gokwik PPCOD%' etc.) that no index can speed up — the fix for that is a MySQL generated column (auto-computed by the database on every write, no separate job, no pipeline code involved) plus an index on it, both applied directly to the schema. This repo's analytics/pipeline.py is a reference copy only — the live ETL runs in a separate repo maintained outside this session, so nothing pipeline-side was or needs to be touched for this fix. The order-count side of Payment Split (COD/Prepaid counts) does already partially exist in the hour_wise_sales rollup, missing only a Partial/PPCOD count column — that piece, if wanted, is genuinely pipeline work for the separate repo. The sales-amount split (₹ COD/Prepaid/Partial) has no rollup coverage at any grain today and was discussed as a future pipeline addition, but is not required for the generated-column fix to work.

Extend the existing session/product rollups to cover more of Product Conversion

PIPELINE, LOWER PRIORITY

Part of this feature already reads from a materialized daily table (mv_product_sessions_by_path_daily), which is good — that part of the pipeline is already doing its job. But two other pieces still get recalculated from scratch on every request: order totals per product, and checkout-initiated events per product, each re-summed over a 60-day window every time someone loads the page. Rolling these up daily the same way would remove that repeated work.

```
analytics/services/productConversionService.js:242 orders_60d CTE
```

```
analytics/services/productConversionService.js:262 ci_60d CTE
```

```
analytics/services/productConversionService.js:271 sessions_60d CTE (already partly rolled up)
```

One thing to design carefully before building any of these

A rollup table is only useful if it's broken down finely enough to answer every filter combination the dashboard actually supports (by UTM source, device, city, product, etc.) — otherwise a filtered request just falls back to scanning the raw tables anyway, and the new table doesn't help the cases that need it most. This needs to be scoped against the real filter options before writing any pipeline code — it's a bigger, slower-to-build fix than the query-level changes above, and worth planning out properly rather than adding on the side.

Internal engineering status report — DATUM backend performance investigation. Code references point to file paths and line numbers current as of this report's preparation date; line numbers may shift as the code changes. Section 1 items 1-7 reflect the original audit; items 8-11 and the updated statuses on items 4-6 were added 2026-08-31 after live verification with Docker access restored.